
SCHOOL OF ENGINEERING AND TECHNOLOGY

**COURSEWORK FOR THE
BSC (HONS) INFORMATION TECHNOLOGY
BSC (HONS) COMPUTER SCIENCE
BSC (HONS) INFORMATION TECHNOLOGY (COMPUTER NETWORKING AND
SECURITY)
BSC (HONS) SOFTWARE ENGINEERING**

ACADEMIC SESSION 2026; SEMESTER 2,3

PRG1203: OBJECT ORIENTED PROGRAMMING FUNDAMENTALS

DEADLINE: 11 AUGUST 2026 11:59PM (Tuesday)

INSTRUCTIONS TO CANDIDATES

- This assignment will contribute 20% to your final grade.
- This is a group (maximum 5 students) assignment

IMPORTANT

The University requires students to adhere to submission deadlines for any form of assessment. Penalties are applied in relation to unauthorized late submission of work.

Any work submitted after the deadline, or after any period of extension granted shall be marked as a Fail or awarded a zero.

Academic Honesty Acknowledgement

"I,,, (student name).
verify that this paper contains entirely my own work. I have not consulted with any outside
person or materials other than what was specified (an interviewee, for example) in the
assignment or the syllabus requirements. Further, I have not copied or inadvertently copied
ideas, sentences, or paragraphs from another student. I realize the penalties (*refer student
handbook undergraduate programme*) for any kind of copying or collaboration on any
assignment."

.....
(Student's signature / Date)

Group Number: _____

Team Members:

No	Name	Student ID
1		
2		
3		
4		
5		

Marking Scheme

Criteria			Marks	Remarks
<u>OOP Design Quality & UML (10%)</u> Implement good object-oriented design in solving the problem, with high modularity, maintainability and reusability. Able to identify appropriate classes and their relationships, complete the classes with appropriate attributes and methods. Correct application of the aggregation/composition, inheritance and polymorphism concepts. The design is well presented in UML diagrams. Demonstrates the ability to create UML class diagrams with correct syntax, including clear and accurate representations of classes and their relationships. UML diagrams are complete, well-structured, and detailed, effectively communicating the system design.	9-10	Excellent		
	7-8	Good		
	5-6	Satisfactory		
	3-4	Poor		
	1-2	Very Poor		
<u>Code Quality (5%)</u> The coding is aligned to the design presented in UML. Follow the best programming practices, such as naming convention, indenting, code structure, optimisation, with appropriate exception handling. Good user-friendliness.	5	Excellent		
	4	Good		
	3	Satisfactory		
	2	Poor		
	1	Very Poor		
<u>Documentation & Presentation (5%)</u> Clear, complete, well-written documentation; Confident, clear explanation; structured slides; effective demo.	5	Excellent		
	4	Good		
	3	Satisfactory		
	2	Poor		
	1	Very Poor		

Objective

By completing this assignment, students will be able to:

- Apply core OOP principles in the design and implementation of a software system.
- Develop a complete Java console-based application.
- Demonstrate the use of inheritance, aggregation/composition, and polymorphism.
- Produce clear and accurate UML class diagrams.
- Follow good programming practices, including coding conventions and documentation standards.

Assignment Requirements

This assignment requires you to design and develop a Java console-based application that demonstrates key Object-Oriented Programming (OOP) concepts. You are free to choose **any topic or domain**—including but not limited to education, business, entertainment, health, lifestyle, productivity, simulation, or games. The chosen application should be sufficiently complex to showcase strong OOP design, including inheritance, aggregation/composition, and polymorphism.

Your Java console application must satisfy the following minimum requirements:

Class Design & OOP Principles

- The application should include **an appropriate number of well-designed classes** with clear responsibilities, demonstrating good object-oriented design principles.
- **Encapsulation** should be implemented using appropriate access modifiers and getter/setter methods where necessary.
- Demonstrate appropriate use of **inheritance** through a well-designed class hierarchy.
- Use **aggregation** and/or **composition** to represent meaningful relationships between classes.
- Demonstrate **polymorphism** through method overriding and/or method overloading.

Functionality & Interaction

- A working **console-based interface** that allows users to interact with the system. (A GUI interface is optional.)
- Functional application logic that is relevant to the chosen topic.
- Basic error handling, including exception handling (e.g., try/catch) and input validation, to improve system robustness.

Documentation & Deliverables

☐ Presentation Video	A short video (no longer than 10 minutes) demonstrating the system's functionality and explaining the Object-Oriented Programming concepts implemented in the system. The video may be presented by one or more team members (as decided by the team lead) and must include the presenter's face with the camera on. Upload the video to YouTube and include the URL in the report.
☐ Report	Report should include: • A cover page with the team group ID, team members' names, and student IDs. • System Description: A clear explanation of the application and the functionality implemented. • OOP Concepts Used: A description of how OOP principles (encapsulation, inheritance, aggregation/composition, polymorphism) are applied in your system. • UML Diagram: Class and class-relationship diagrams showing classes, attributes, methods, relationships, and multiplicities. • Presentation Video URL
☐ Code	Well-commented Java source code following standard coding conventions.

IMPORTANT: You are required to zip the report and the project source code (the entire project folder) into a single ZIP file and upload it to eLearn. Only the team leader is required to make the submission.

Guideline

Getting Started: When embarking on an object-oriented programming project, it's essential to follow these steps, prioritizing design before coding:

1. Identify the classes and their attributes, e.g. identify all the potential model or controller class.
2. Determine the class relationships, e.g. composition, aggregation, and inheritance.
3. Create a UML diagram to visualize your design.
4. Initiate the coding phase.

Evaluating Object-Oriented Design: To ensure that your program exhibits a strong object-oriented design, always verify your design against the following criteria:

Criteria of Quality Object-Oriented Design and Code:

- **Modularity:** The program is divided into classes and objects, with each class having a clear and distinct responsibility. This promotes code reusability and maintainability.
- **Encapsulation:** Classes encapsulate their data (attributes) and behavior (methods), restricting direct access to internal state. Access to data is controlled through getter and setter methods.
- **Inheritance:** Inheritance is used when a class shares common attributes and behavior with another class. It allows for code reuse and the creation of specialized classes based on a common base.
- **Polymorphism:** Polymorphism enables objects of different classes to be treated as objects of a common superclass. It allows for flexibility and dynamic method dispatch.
- **Loose Coupling:** Objects are loosely coupled when they interact with each other through well-defined interfaces. This reduces the interdependence of components, making the system more flexible and maintainable.
- **High Cohesion:** Classes and modules should have a single, well-defined purpose, with methods and attributes that are closely related. This minimizes the need for excessive dependencies between classes.
- **Single Responsibility Principle (SRP):** Each class should have only one reason to change, meaning it should have a single responsibility. This principle leads to more maintainable and understandable code.
- **Code Reusability:** Reusable components (classes, libraries, and modules) are designed to minimize redundancy and promote efficient code reuse.
- **Consistent Naming and Coding Conventions:** Adherence to consistent naming conventions and coding styles improves the readability and maintainability of the codebase.
- **Flexibility and Scalability:** Object-oriented design should allow the program to evolve and scale over time by accommodating new features and changes without major overhauls.